

MIPS Reverse Engineering

MIPS32, the O32 ABI, and other MIPS Flavors

Why reverse engineer MIPS at all?

What all is targeted in actual attacks?

Network edge / perimeter appliances

- Enterprise firewalls, routers, load balancers
- VPN gateways
- Consumer routers

Virtualization infrastructure

- ESXi, vCenter, vSphere servers
- Citrix gateways

Cloud workloads and orchestration

- Kubernetes
- Containers, registries, and CI/CD systems

Storage and backup systems

- NAS devices
- Backup servers
- Cloud storage

Industrial technology

- PLCs and ICS controllers
- Security cameras

Miscellaneous

- UEFI/BIOS and other firmware
- Baseboard management (BMCs)
- Mobile devices
- Smart home devices
- Printers

GreyNoise State of the Edge Report 2026

Where Your Edge Defenses Fall Short

GreyNoise measured 212 exploitation attempts per second across H2 2025 — and the patterns inside that volume expose specific, measurable gaps in common edge defense strategies.

The *2026 GreyNoise State of the Edge Report* analyzes 2.97 billion malicious sessions from 3.8 million unique source IPs. It shows exactly which vendors are targeted, where attack infrastructure concentrates, where reputation-based defenses have coverage gaps, and what defenders can do about it.

The Verizon 2025 DBIR documented an 8x increase in edge device exploitation — edge vulnerabilities jumped from 3% to 22% of all vulnerability exploitation breaches. Mandiant M-Trends 2025 found the top four most frequently exploited vulnerabilities were all in edge devices — Palo Alto PAN-OS, Ivanti Connect Secure, Ivanti Policy Secure, and Fortinet FortiClient EMS. CISA issued Binding Operational Directive 26-02, requiring federal agencies to address end-of-support edge devices. This report puts numbers behind those findings.

What's Inside

- **VPN Targeting at Scale** — Palo Alto GlobalProtect: 16.7 million sessions, more than 3.5x Cisco and Fortinet combined. Vendor-by-vendor breakdown of what's absorbing the most traffic.
- **The Reputation Gap** — 52% of remote code execution attempts came from IPs appearing in GreyNoise for the first time — infrastructure so new that no threat feed had it yet. GreyNoise caught them on first contact. Why static blocklists alone aren't enough.
- **A 300,000-IP Residential Botnet** — Grew from 2,000 to 300,000 IPs in 72 days. 73% residential. Why traditional perimeter controls miss it.
- **Legacy CVE Exploitation** — Pre-2015 CVEs generated 7.3 million sessions — 4x more than 2023-2024 CVEs, with one 26-year-old vulnerability (CVE-1999-0526) accounting for the majority. Why the long tail of old vulnerabilities creates persistent exposure.
- **AI Infrastructure: Emerging Target** — 91,403 sessions targeting LLM inference servers. 175,000 Ollama instances identified as internet-exposed (SentinelLABS/Censys, January 2026). The newest attack surface at the edge.

<https://www.greynoise.io/resources/2026-state-of-the-edge-report>

Driving Factors

Heavily protected endpoints

EDR-blindness almost everywhere else

Lack of vulnerability management

Identity and tokens are now device's biggest export

End-of-life exploitation

Analysis Challenges

Abstraction lifting:

What is defined by the CPU?

What is defined by the compiler and the ABI?

And the OS?

Automated analysis: Decompilers, binary lifters, IR translation

Human-centered analysis:

The cognitive and analytical process of moving from concrete binary representations up through layers of abstraction until you reach **architecture-independent, intent-revealing understanding**

The Cornerstones of RE Knowledge Transfer

Basic OS architecture

File system, system configuration, networking, built-in defense mechanisms, etc. etc.

File formats, process execution and multi-threading

Linkers and loaders, specific evasion tricks, dynamic analysis challenges, OS-specific data structures

Assembly language

Basic arithmetic, memory access patterns, specifics like delay slots or barrel shifters

Calling conventions

Data flow tracking depends on these conventions

Compiler convention and architectural requirements

Programming language / compiler intricacies

Stack and memory management, optimization, libraries and symbol recovery

Linux OS Reverse Engineering Tidbits

Everything is a file

Standard file system structure

/proc file system holds system configuration

System library landscape a lot simpler

ELF file format and mostly System-V calling convention

Primary execution unit is the process, not threads

Threat landscape by far less advanced

Threat detection hardly present

ELF: The Executable & Linkable Format

Standard file format for executable files, relocatable objects and shared libraries on Unix platforms

Flexible, extensible, cross-platform

Native analysis tools available

ELF Highlights:

Sections

Segments

Symbols

Relocation

Linking and loading

Dynamic linking

```
ELF Header:
Magic:  7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
Class:                                     ELF64
Data:                                       2's complement, little endian
Version:                                  1 (current)
OS/ABI:                                    UNIX - System V
ABI Version:                              0
Type:                                      DYN (Shared object file)
Machine:                                  Advanced Micro Devices X86-64
Version:                                  0x1
Entry point address:                      0x0
Start of program headers:                 64 (bytes into file)
Start of section headers:                181040 (bytes into file)
Flags:                                    0x0
Size of this header:                      64 (bytes)
Size of program headers:                  56 (bytes)
Number of program headers:                12
Size of section headers:                  64 (bytes)
Number of section headers:                31
Section header string table index: 30
```

```
Section Headers:
[Nr] Name                               Type      Address                               Offset
     Size                               EntSize   Flags  Link  Info  Align
[ 0]                                     NULL      0000000000000000                     0 0 0
     0000000000000000 0000000000000000                     0 0 0
[ 1] .note.gnu.pr[...]                   NOTE      00000000000002e0                     0 0 8
     0000000000000020 0000000000000000      A      0 0
[ 2] .note.gnu.bu[...]                   NOTE      0000000000000300                     0 0 4
     0000000000000024 0000000000000000      A      0 0
[ 3] .note.package                       NOTE      0000000000000324                     0 0 4
```

Linux 64-bit: System-V Calling Convention

Parameters to functions are passed in via the registers

rdi, rsi, rdx, rcx, r8, and r9 (in that order!)

Floating-point parameters are passed in via xmm0 through xmm7

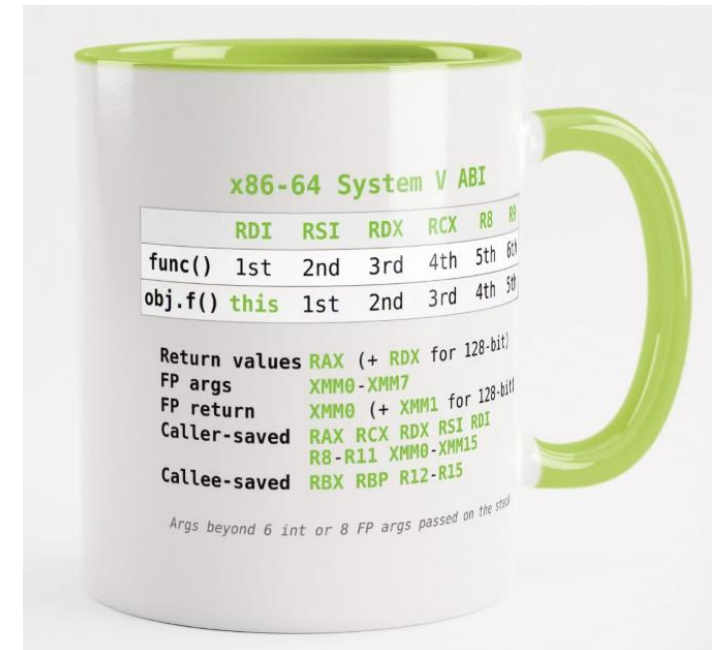
Functions preserve the registers **rbx, rsp, rbp, r12, r13, r14, r15**

rax, rdi, rsi, rdx, rcx, r8, r9, r10, r11

are scratch registers

Return value goes in **rax**, or **rax:rdx** if bigger than 64-bit

In C++ binaries the *this* pointer is handed over via **rdi**



MIPS – Brief History



MIPS is a classic **RISC architecture**

Designed around simplicity and regularity, easy to learn

Development started in **1981 at Stanford**

Evolved through MIPS I-V (1985-96) then unified into
MIPS32/MIPS64 (1999-2014)

Widely used in legacy systems, PlayStation 1&2, Nintendo 64

Development ceased in 2021 when owner pivoted to RISC-V

Still **very much alive** in deployed hardware – routers, IoT devices,
legacy embedded systems, industrial systems

The MIPS Pipeline

MIPS has a 5-stage in-order pipeline, each stage equaling one clock cycle

- Instruction fetch
- Instruction decode
- Execute
- Memory access
- Write back

Cycles	1	2	3	4	5	6	7
Instruction 1	IF	ID	EX	ME	WB		
Instruction 2		IF	ID	EX	ME	WB	
Instruction 3			IF	ID	EX	ME	WB

As such, 3 instructions would take 7 cycles, not 15 as it would otherwise

This parallelism also means that the next instruction is always already in-flight

Poses an issue for branch and load instructions, hence branch delay slots and load delay slots

Branch / Load Delay Slots

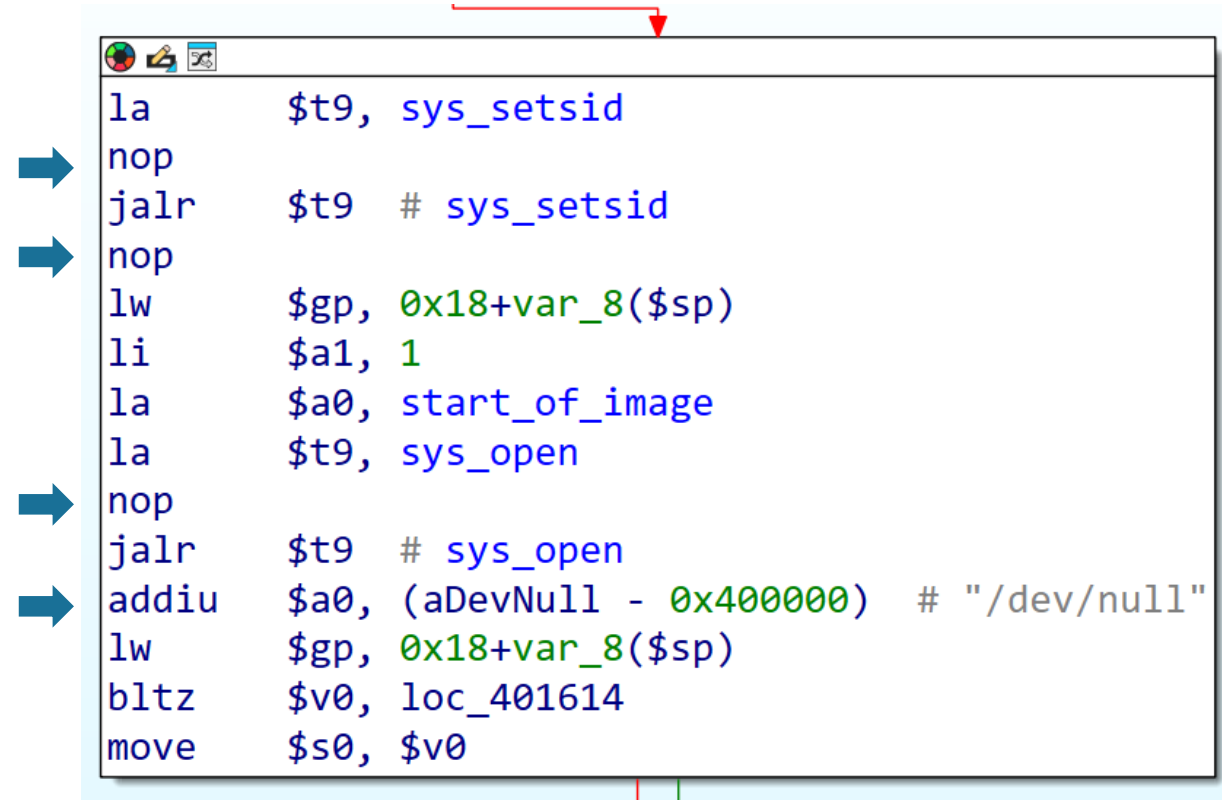
When the CPU fetches a branch instruction, by the time it reaches the EX stage, the **next instruction is already being computed**

The instruction after a branch **will always execute** – called branch delay slot

Same effect on load instructions

Compilers try to place needed instructions there or pad it with nop

MIPS32r6 abolished delay slots



The diagram shows a MIPS assembly code snippet with several instructions. A red arrow at the top points to the first instruction, `la $t9, sys_setsid`. To the left of the code, blue arrows point to the instructions `jalr $t9 # sys_setsid`, `lw $gp, 0x18+var_8($sp)`, `jalr $t9 # sys_open`, and `addiu $a0, (aDevNull - 0x400000) # "/dev/null"`. The code is as follows:

```
la      $t9, sys_setsid
nop
jalr    $t9 # sys_setsid
nop
lw      $gp, 0x18+var_8($sp)
li      $a1, 1
la      $a0, start_of_image
la      $t9, sys_open
nop
jalr    $t9 # sys_open
addiu   $a0, (aDevNull - 0x400000) # "/dev/null"
lw      $gp, 0x18+var_8($sp)
bltz    $v0, loc_401614
move    $s0, $v0
```

Instruction Categories

R-Type (Register)

All operands are registers, used for arithmetic, logic, shifts, jr, syscall, etc.

Example: `add $t0, $t1, $t2`

I-Type (Immediate)

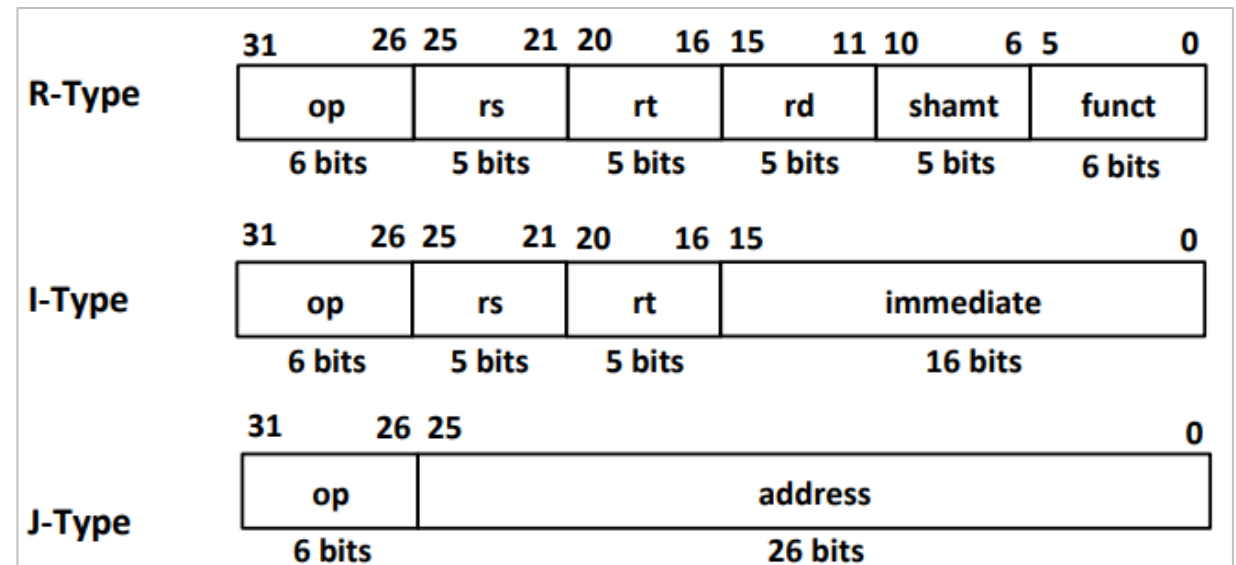
One register, one immediate operand, used for loads, stores, branches, addi, lui, etc.

Example: `lw $t0, 8($sp)` (SP+8)

J-Type (Jump)

One 26-bit jump target address, shifted left by 2 and combined with upper bits of PC; only j and jal instructions use this format

Example: `jal sub_402DA0`



Key Instruction Groups

Arithmetic: *add, addu, sub, addi, addiu* (signed vs. unsigned affects overflow trapping, not the bit pattern)

Logic: *and, or, xor, nor, andi, ori*

Shifts: *sll, srl, sra, sllv, srlv, srav* (fixed vs. variable shift)

Memory: *lw, sw, lb, lb, lh, sh, lui* (note: no memory-to-memory operations)

Comparison: *slt, sltu, slti* – sets register to 1 or 0, feeds branches like x86 *cmp*

Branches: *beq, bne, bgtz, bltz, bgez*, etc.

Jumps: *j, jal, jr \$ra (return), jalr*

MIPS Registers and their Purpose

Register	Name	Purpose
\$0	\$zero	Always zero — hardwired. Writes are discarded
\$1	\$at	Assembler temporary — do not rely on preservation
\$2–\$3	\$v0–\$v1	Return values / syscall number in \$v0
\$4–\$7	\$a0–\$a3	First four function arguments
\$8–\$15	\$t0–\$t7	Temporaries — caller-saved, not preserved across calls
\$16–\$23	\$s0–\$s7	Saved registers — callee-saved, must be preserved
\$24–\$25	\$t8–\$t9	More temporaries ; \$t9 used for PIC indirect calls
\$26–\$27	\$k0–\$k1	Kernel reserved — OS interrupt handlers only
\$28	\$gp	Global pointer — points to global/static data in PIC code
\$29	\$sp	Stack pointer
\$30	\$fp / \$s8	Frame pointer (when used) or extra saved register
\$31	\$ra	Return address — written by jal/jalr

Not architecturally defined; the MIPS stack is a software concept, ABI driven

No push/pop instructions — the compiler explicitly adjusts \$sp and uses sw/lw to save/restore registers

Stack pointer (\$sp) must **remain aligned** at all times

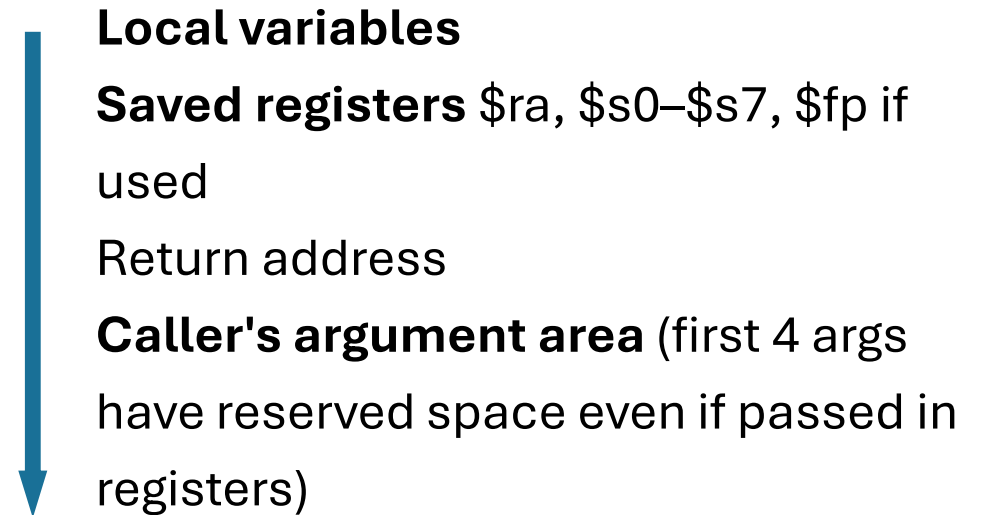
The stack frame is **allocated in one instruction** at the function prologue

Grows downward

The 16-byte **argument home**: even if a function takes no arguments, O32 requires the caller to reserve 16 bytes of stack space for the callee

The MIPS Stack

Stack frame layout (O32):



The O32 Calling Convention

Argument passing: **\$a0-\$a3**, then stack; \$f12/14 for floats

Return values: **\$v0** or \$v0 and \$v1 for 64-bit values, \$f0 for float

Caller-saved registers: \$t0-\$t9, \$a0-\$a3, \$v0-\$v1

Callee-saved registers: \$s0-\$s7, \$fp, \$ra, \$gp

The \$ra pattern

\$ra set by calling instruction, must be callee-saved if further calls happen

The global pointer \$gp

In position independent code \$gp is used to locate static global data

The \$t9 convention

In position independent code \$t9 holds the target address of indirect calls

```
Set up $gp [ li      $gp, (got+0x7FF0 - .)
              addu    $gp, $t9
Set up stack frame [ addiu   $sp, -0x128
                    sw      $ra, 0x118+var_sC($sp)
                    sw      $s2, 0x118+var_s8($sp)
                    sw      $s1, 0x118+var_s4($sp)
                    sw      $s0, 0x118+var_s0($sp)
                    sw      $gp, 0x118+var_108($sp)
Call memset [ la        $t9, memset
              addiu    $s2, $sp, 0x118+var_100
              move     $a0, $s2
              move     $a1, $zero
              jalr     $t9 # memset
              li       $a2, 0x100
```

MIPS Syscalls

MIPS syscalls follow a consistent convention, but syscall numbers differ by OS and ABI

Syscall number goes into \$v0 before syscall instruction

Arguments are passed in \$a0-\$a3

Return comes back in \$v0 (\$a3 holding the error flag, if any)

Syscall number tables differ by:

OS flavor

ABI – O32, N32 or N64 with bases 4000, 6000, or 5000 respectively

O32 Linux table:

<https://github.com/luapower/syscall/blob/master/syscall/linux/mips/nr.lua>

Lab – Cross-Architecture Analysis

AcidRain Wiper from ViaSAT hack

Please work with the provided .bndb with core libc functions recovered

If no Binja license available, work with BinaryNinja Cloud, you can upload the .bndb instead of the sample

9B4DFACA873961174BA935FDDAF696145AFE7BBF5734509F95FEB54F3584FD9A

Handy materials

<https://www.kth.se/social/files/563c63c9f276547044e8695f/mips-ref-sheet.pdf>

<https://github.com/luapower/syscall/blob/master/syscall/linux/mips/nr.lua>

Further reading

<https://bh-cookbook.github.io/mips-asm/learning-mips-assembly-part-1.html>